

Лабораторная работа №1

Установка и конфигурация операционной системы на виртуальную машину

Павлова Татьяна Юрьевна

2026-09-04

1 Информация

Раздел 1

Информация

- Павлова Татьяна Юрьевна

- Павлова Татьяна Юрьевна
- студентка НКАбд-04-24

- Павлова Татьяна Юрьевна
- студентка НКАбд-04-24
- Российский университет дружбы народов им. П. Лумумбы

- Павлова Татьяна Юрьевна
- студентка НКАбд-04-24
- Российский университет дружбы народов им. П. Лумумбы
- 1132246745@rudn.ru

- Павлова Татьяна Юрьевна
- студентка НКАбд-04-24
- Российский университет дружбы народов им. П. Лумумбы
- 1132246745@rudn.ru
- https://github.com/tanuha228/study_2025-2026_infosec-intro

Освоить на практике применение режима однократного гаммирования

Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно: 1. Определить вид шифротекста при известном ключе и известном открытом тексте. 2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста.

Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, листинг программы и результаты выполнения приведены в отчете.

Требуется разработать программу, позволяющую шифровать и дешифровать данные в режиме однократного гаммирования. Начнем с создания функции перевода текстовых символов в числовые значения (рис. [-@fig:001]).

```
import random

def text_to_codes(text):
    """Преобразует строку в список кодов символов"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes
```

Рисунок 1: Функция перевода текста в числа

Необходимо определить вид шифротекста при известном ключе и известном открытом тексте. Так как операция исключающего или отменяет сама себя, делаем одну функцию и для шифрования и для дешифрования текста (рис. [-@fig:002]).

```
def xor(b1, b2):  
    """XOR двух списков кодов"""  
    if len(b1) != len(b2):  
        print("Ошибка: длины не соответствуют")  
        return None  
    result = []  
    for i in range(len(b1)):  
        new = b1[i]^b2[i]  
        result.append(new)  
    return result
```

Рисунок 2: Функция для шифрования текста

Также создаю функцию преобразования чисел обратно в текст (рис. [-@fig:003]).

```
def codes_to_text(codes):  
    """Преобразует список кодов обратно в строку"""  
    text = ""  
    for i in range(len(codes)):  
        text += chr(codes[i])  
    return text
```

Рисунок 3: Подбор возможных ключей для фрагмента

Добавляю функцию генерации случайного ключа (рис. [-@fig:004]).

```
def generate_key(length):  
    """Генерирует случайный ключ заданной длины"""  
    key = []  
    for i in range(length):  
        key.append( random.randint(0, 255))  
    return key
```

Рисунок 4: Генерация ключа

Проверка работы всех функций. Шифрование и дешифрование происходит верно, как и нахождение ключей, с помощью которых можно расшифровать верно только кусок текста (рис. [-@fig:005]). (рис. [-@fig:006]).

```
# основная программа
plain = ("С Новым Годом, друзья!")
print(f"Открытый текст: {plain}")

# генерация ключа
key = generate_key(len(plain))
print(f"Ключ: {key}")

# преобразование ключа в коды
plain_codes = text_to_codes(plain)

# шифрование
cipher_bytes = xor(plain_codes, key)
cipher_text = codes_to_text(cipher_bytes)
print(f"Зашифрованный текст: {cipher_text}")

# дешифровка
decrypted_bytes = xor(cipher_bytes, key)
decrypted_text = codes_to_text(decrypted_bytes)
print(f"Расшифровано: {decrypted_text}")

# поиск ключа
target = ("С Новым Годом, друзья!")
```

```
[tanya@user lab07]$ python3 lab1.py
Открытый текст: С Новым Годом, друзья!
Ключ: [5, 5, 118, 18, 104, 167, 42, 47, 133, 203, 22, 125, 197, 184, 175, 40, 212, 119, 121, 98, 168, 233]
Зашифрованный текст: Ф%ъньЭЖЖЎТуйМБдюЮбЕ
Расшифровано: С Новым Годом, друзья!
Искомый текст: С Новым Годом, друзья!
Найденный ключ: [5, 5, 118, 18, 104, 167, 42, 47, 133, 203, 22, 125, 197, 184, 175, 40, 212, 119, 121, 98, 168, 233]
```

Рисунок 6: Результат работы программы

В ходе выполнения данной лабораторной работы, мною было освоено на практике применение режима однократного гаммирования.